

EFBY Request Lab

Cliente API nativo para macOS con compatibilidad Postman, orquestación BPMN y sincronización Git

Swift 6.3

Clean Architecture

SDD + TDD

166 tests

DMG notarizado

Autor: Raúl Rodríguez · 2026

El problema

- Los equipos dependen de clientes cloud (Postman, Insomnia) para probar APIs.
- Es difícil versionar colecciones junto al código en Git sin fricción.
- El Collection Runner es lineal: sin ramas condicionales ni paralelismo visual.
- Falta una herramienta nativa macOS/iPad integrada en el ecosistema Swift del equipo.

La solución

EFBY Request Lab es un laboratorio API nativo que ofrece:

- HTTP y WebSocket con variables por capas
- Scripts JavaScript compatibles con `pm.*`
- Import/export Postman y OpenAPI
- Workspace local + sync Git manual
- Flujos BPMN con gateways y paralelismo

Usuarios objetivo

- **Desarrollador backend** — endpoints y scripts
- **QA / integración** — colecciones y flujos
- **DevOps** — colecciones en repo Git
- **Equipo EFBY** — herramienta offline

Diferenciadores vs Postman

Aspecto	Postman	EFBY Request Lab
Plataforma	Electron multi-OS	Nativo Swift/SwiftUI macOS + iPad
Sincronización	Cloud para equipos	Git manual, control total
Flujos	Collection Runner lineal	BPMN con gateways, paralelismo y timers
Offline	Limitado	Workspace 100 % local
Código	Cerrado	Open source en GitHub

Funcionalidades principales

- **HTTP** — editor, auth, TLS, scripts, historial
- **WebSocket** — transcript, ping, scripts
- **Variables** — global, colección, entorno y local
- **Postman** — Collection v2.0/v2.1 y entornos
- **OpenAPI** — importación 3.0/3.1 JSON
- **Git** — pull/push manual de colecciones
- **BPMN** — editor visual y ejecución de flujos
- **Scripts** — `pm.crypto` RSA/AES
- **iPad** — variante `EfbyPostmanPad`
- **Sin login** — app de escritorio offline

Stack tecnológico

Componente	Tecnología
Lenguaje	Swift 6.3 (conurrencia estricta)
UI	SwiftUI + AppKit (macOS) / SwiftUI (iPad)
Paquetes	Swift Package Manager
Red	URLSession, WebSocket nativo
Scripts	JavaScriptCore (pm.*)
Persistencia	JSON local + Git manual
Flujos	BPMN en WebView embebido
CI/CD	GitHub Actions + DMG notarizado

Clean Architecture en capas SPM

```
EfbyRequestLabs / EfbyPostmanPad (UI SwiftUI)
    ↓
EfbyPresentation (MainViewModel, coordinadores, AppDependencies)
    ↓
EfbyApplication (Ports + Use Cases)
    ↓
EfbyDomain (modelos)
    ↑
EfbyInfrastructure (implementa Ports)
```

Domain no importa SwiftUI. Application no usa URLSession directamente: solo vía ports en Infrastructure.

Targets SPM y responsabilidades

Target	Responsabilidad
EfbyDomain	Modelos de dominio puros
EfbyApplication	Puertos y casos de uso
EfbyInfrastructure	HTTP, WS, Git, scripts, codecs, repos
EfbyPresentation	ViewModel, coordinadores, composition root
EfbyRequestLabs	Aplicación macOS
FlowDebugRunner	CLI de depuración de flujos

Coordinadores de presentación

- `GitWorkspaceCoordinator`
- `GitSessionCoordinator`
- `RequestTabCoordinator`
- `RequestTabsCoordinator`
- `WebSocketExecutionCoordinator`
- `FlowExecutionCoordinator`
- `WorkspacePersistenceCoordinator`
- `DocumentImportCoordinator`
- `SharedWorkspaceCoordinator`
- `EnvironmentCoordinator`
- `WorkspaceCatalogCoordinator`
- `BitbucketPadCoordinator`

El `MainViewModel` actúa como fachada UI; la lógica de dominio vive en coordinadores y casos de uso.

Ejecución HTTP (ejemplo)

1. La vista llama a `MainViewModel.sendCurrentRequest()`.
2. `RequestTabCoordinator` **invoca** `ExecuteHTTPRequestUseCase`.
3. El use case usa `HTTPExecutionServiceProtocol`.
4. `RequestExecutionService` ejecuta la petición y los scripts.
5. El `ViewModel` aplica variables y persiste vía `WorkspacePersistenceCoordinator`.

Tres capas de datos

1. Local

`WorkspaceRepository` guarda el workspace en Application Support: pestañas, historial, configuración.

2. Workdir Git

`SharedCollectionsRepository` gestiona colecciones, entornos y flows en el directorio compartido.

3. Remoto

Pull/push manual con `GitSessionCoordinator` y `GitWorkspaceCoordinator`. Sin backend propio ni sync cloud.

Specification-Driven Development (SDD)

Las specs son la fuente de verdad funcional antes de implementar el código.

- **10 especificaciones** en `SDD/tile/specs/*.spec.md`
- Cada spec define comportamiento, criterios de verificación y trazabilidad
- IDs `REQ-*` enlazados a tests en la matriz de trazabilidad
- Reglas de desarrollo y seguridad en `SDD/tile/rules/`

Las 10 especificaciones SDD

Spec	Área
http-request-execution	Pipeline HTTP completo
variable-resolution	Precedencia de variables
javascript-pm-runtime	Runtime pm.* y criptografía
websocket-client	Cliente WebSocket
workspace-flow	Flujos BPMN
postman-interop	Import/export Postman y OpenAPI
git-workspace	Operaciones Git
persistencia-workspace	Persistencia JSON y migraciones
main-view-model-workspace	Coordinador de workspace
javascript-tooling	Editor y autocompletado JS

Test-Driven Development (TDD)

Ciclo **Red** → **Green** → **Refactor** aplicado en todo el proyecto.

- **166 tests** automatizados en `AppCoreTests`
- **36 requisitos REQ-*** con trazabilidad spec → test
- Tests deterministas con red simulada (`URLProtocol`)
- CI ejecuta `swift test` en cada push

Suites destacadas

- `RequestExecutionServiceTests`
- `WorkspaceFlow*Tests`
- `PostmanCollectionCodecTests`
- `GitRepositoryServiceTests`
- `ScriptEngineTests`

Architecture Decision Records (ADR)

ADR	Decisión
ADR-001	SPM con módulos compartidos macOS/iPad
ADR-002	Clean Architecture + MVVM en presentación
ADR-003	JavaScriptCore para runtime Postman parcial
ADR-004	Persistencia JSON local + Git manual
ADR-005	BPMN como orquestación de flujos API

CI/CD y distribución

GitHub Actions

- `ci.yml` — build + tests en macOS 26 / Swift 6.3
- `release.yml` — tag `v*` genera DMG
- Firma Developer ID + notarización Apple

Instalación

1. Descargar DMG desde GitHub Releases
2. Arrastrar a Applications
3. Ejecutar sin bloqueo de Gatekeeper

```
swift build && swift test  
./Tools/build_dmg.sh --sign --notarize
```


Estructura del proyecto

```
Sources/  
  AppCore/Domain/      → EfbyDomain  
  AppCore/Application/ → EfbyInfrastructure  
  EfbyApplication/     → Ports + Use Cases  
  EfbyPresentation/    → ViewModel + Coordinadores  
  EFBYPostman/         → App macOS  
Tests/AppCoreTests/    → 166 tests  
docs/                 → Documentación y ADRs  
SDD/tile/specs/       → 10 especificaciones  
.github/workflows/     → CI y releases
```

Enlaces y cierre

Repositorio: github.com/efby/TMF_EFBY_REQUEST_LAB

DMG notarizado: [GitHub Releases](#)

Vídeo demo: youtu.be/Q15wS9GjTJ0

Documentación: docs/README.md en el repositorio

Cliente API nativo · Offline · Open source · Listo para producción

Gracias